

MATRIZ DE COSTOS Y OPTIMIZACIÓN DE TOKENS

Mesa de Ayuda IT – n8n + OpenRouter + Airtable

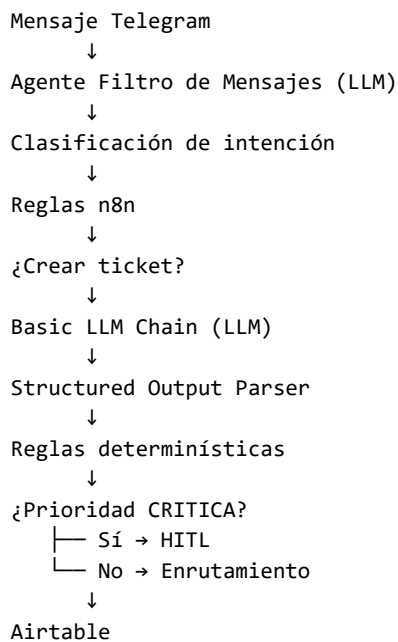
1. Objetivo

Este documento analiza el consumo de procesamiento de IA del workflow de Mesa de Ayuda IT y establece estrategias para reducir tokens, latencia y costo sin degradar la funcionalidad. La matriz se plantea sobre la arquitectura realmente implementada y evita asumir precios comerciales que pueden cambiar según el proveedor, modelo o fecha.

2. Componentes que pueden generar consumo de IA

Componente	Función	Uso de IA	Estrategia
Agente Filtro de Mensajes	Determina intención	Sí	Prompt corto + salida estructurada
Basic LLM Chain	Clasifica el ticket IT	Sí	Prompt especializado + JSON estructurado
Structured Output Parser	Valida estructura	No es un modelo independiente	Limitar campos y valores
Reglas n8n	Enrutamiento, prioridad y duplicados	No	Preferir lógica determinística
Airtable	Persistencia	No	Sin consumo de tokens
Telegram	Entrada/salida	No	Sin consumo de tokens
HITL	Aprobación humana	No	Evita razonamiento adicional de IA

3. Arquitectura actual de procesamiento



La arquitectura evita enviar a la segunda etapa mensajes que no requieren ticket. Esto es importante porque SALUDO, CONVERSACION y NO_VALIDO no necesitan clasificación IT completa.

4. Variables que impactan el consumo

Variable	Impacto	Control implementado
Longitud del mensaje del usuario	Aumenta tokens de entrada	Se procesa el mensaje recibido, sin contexto histórico innecesario
Prompt del sistema	Aumenta tokens de entrada en cada llamada	Prompts específicos y sin contenido redundante
Cantidad de campos de salida	Aumenta tokens de salida	Structured Output Parser con esquema limitado
Max Tokens	Define techo de salida	500 en los nodos LLM actuales
Cantidad de llamadas LLM	Impacto principal en costo	Filtro previo + reglas determinísticas
Reintentos	Puede multiplicar consumo	Validaciones y control de errores
Historial conversacional	Puede aumentar considerablemente entrada	No se envía historial completo al clasificador

5. Matriz de optimización

Estrategia	Antes / riesgo	Implementación	Beneficio esperado
Filtro de intención	Clasificar todo con el modelo principal	Agente Filtro de Mensajes	Evita clasificación IT innecesaria
Salida JSON estricta	Texto libre	Structured Output Parser	Reduce respuestas extensas y errores
Catálogos cerrados	Valores libres	Enum de categorías/prioridad/impacto/urgencia	Menos tokens y menor ambigüedad
Reglas determinísticas	Delegar decisiones simples a IA	Switch / IF / expresiones n8n	Sin consumo de IA
HITL	Reprocesar automáticamente casos críticos	Aprobación por Telegram	Control humano sin llamadas adicionales
Duplicados antes de IA	Procesar mensajes repetidos	Buscar Ticket Duplicado	Evita consumo repetido
Prompt especializado	Prompt genérico y extenso	Prompt orientado a Mesa de Ayuda	Menor contexto innecesario
Max Tokens controlado	Salida sin límite práctico	500 tokens	Protección ante respuestas excesivas

6. Matriz de costo por ejecución

La siguiente matriz no fija un precio monetario universal porque el costo real depende del precio vigente del modelo seleccionado en OpenRouter y de su modalidad de cobro. Se utiliza una unidad normalizada de tokens para poder comparar escenarios.

Etapas	Llamadas	Entrada	Salida	Costo relativo
Filtro de intención	1	Prompt + mensaje	JSON corto	Bajo
Clasificación IT	0 o 1	Prompt + mensaje	JSON estructurado	Bajo/medio
Reglas n8n	0	0	0	Nulo
Airtable	0	0	0	Nulo
Telegram	0	0	0	Nulo
HITL	0	0	0	Nulo

7. Modelo de estimación

Para cualquier modelo con precio por tokens, el costo aproximado puede expresarse como:

Costo por ejecución=
(tokens entrada × precio entrada / unidad de precio)
+
(tokens salida × precio salida / unidad de precio)

En la práctica, debe utilizarse la unidad indicada por el proveedor. Para OpenRouter, se recomienda registrar periódicamente el modelo exacto, precio vigente y consumo observado antes de convertir la matriz en un importe monetario.

8. Escenario de referencia

Para comparar alternativas se utiliza un escenario hipotético de 1.000 mensajes recibidos. No representa una factura real; es una base de simulación para demostrar el efecto de la arquitectura.

Tipo de mensaje	Cantidad ejemplo	Filtro IA	Clasificación IT	Llamadas IA
SALUDO / CONVERSACION / NO_VALIDO	300	Sí	No	300
INCIDENTE / SOLICITUD	700	Sí	Sí	1.400
Total	1.000	1.000	700	1.700

El ejemplo considera dos llamadas para los 700 mensajes que continúan hasta clasificación IT. Si se implementara una sola llamada LLM para todos los mensajes, serían aproximadamente 1.000 llamadas. La comparación demuestra que el diseño debe evaluarse considerando tanto cantidad de llamadas como longitud de cada prompt.

9. Optimización propuesta para una futura versión

- Reducir instrucciones repetidas en prompts manteniendo las reglas indispensables.
- Usar un modelo económico para el filtro de intención y reservar un modelo más capaz para casos ambiguos o críticos.
- Aplicar clasificación determinística por palabras clave solamente cuando la regla sea suficientemente confiable.
- Reducir Max Tokens al mínimo que cubra la salida JSON esperada.
- Evitar enviar campos que el modelo no necesita.
- Evitar reenviar historial completo de Telegram.
- Detectar duplicados antes de la clasificación IT.
- Registrar métricas de tokens por ejecución para calcular costo real.
- Revisar periódicamente los prompts y retirar instrucciones redundantes.

10. Matriz de eficiencia

Control	Impacto sobre costo	Impacto sobre calidad	Prioridad de implementación
Filtro de intención	Alto	Positivo	Ya implementado
Duplicados antes de IA	Medio/alto	Positivo	Ya implementado
Structured Output	Medio	Positivo	Ya implementado
Reglas determinísticas	Alto	Positivo	Ya implementado
Reducción de prompt	Medio	Debe validarse	Siguiente optimización
Reducción de Max Tokens	Bajo/medio	Debe validarse	Siguiente optimización
Métricas de tokens	No reduce por sí sola	Mejora control	Recomendado
Modelo económico para filtro	Potencialmente alto	Debe validarse	Futuro

11. Métricas recomendadas

Métrica	Descripción
mensajes recibidos	Cantidad total de mensajes Telegram
Llamadas llm	Cantidad de invocaciones a modelos
Tickets creados	Tickets efectivamente persistidos
Mensajes no validos	Mensajes descartados
Duplicados bloqueados	Mensajes repetidos no procesados
Tokens entrada	Tokens consumidos como entrada
Tokens salida	Tokens generados
Tokens totales	Entrada + salida
Errores llm	Fallos de llamadas al modelo
Costo estimado	Costo calculado con tarifa vigente

12. Relación entre costo y calidad

La reducción de tokens no debe realizarse eliminando reglas esenciales. En esta solución se prioriza una optimización conservadora: reducir contexto innecesario y usar lógica determinística donde no aporta valor el modelo, manteniendo las restricciones de salida y las validaciones. Cualquier cambio de modelo o prompt debe repetirse sobre el conjunto de pruebas funcionales y unhappy paths.

13. Evidencia de optimización aplicada

- Los mensajes NO_VALIDO no llegan a la creación de tickets.
- Los saludos no generan clasificación IT.
- Las conversaciones no generan ticket.
- Los duplicados se detectan por chat_id + telegram_message_id.
- El modelo de clasificación devuelve únicamente los campos requeridos.
- Los catálogos están restringidos mediante salida estructurada.
- Las decisiones de enrutamiento se realizan con Switch/IF.
- Los casos CRITICA utilizan aprobación humana.
- El registro de errores se realiza sin una llamada adicional al LLM.

14. Conclusión

La arquitectura implementada aplica optimización de costos desde el diseño: filtra antes de clasificar, evita reprocesar duplicados, limita la salida de los modelos y delega decisiones determinísticas a n8n. El siguiente nivel de madurez consiste en medir tokens reales por ejecución y utilizar esos datos para ajustar prompts, límites y selección de modelos.